

 This is a copy of a chat between Claude and Bryan. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

I'm looking to refactor this <https://github.com/NCAS-CMS/cf-plot/blob/main/cfplot/contour.py>, into a simple class structure, less duplicated code, and a clear distinction between the viewport with the contour in it, and space around it for annotation

Fetches: cf-plot/cfplot/contour.py at main · NCAS-CMS/cf-plot >

Let me get the raw file to see the full code:

Searched the web >

I can see the first ~1000 lines from the GitHub page. This is a 2580-line monolithic function — quite a beast. Let me give you a thorough refactoring proposal based on what I can see, plus the patterns that are clearly repeated throughout.

Here's my analysis and a proposed class structure:

What's wrong with the current structure

The `con()` function is ~2500 lines doing everything: argument parsing, data extraction, level calculation, colorscale management, map setup, filled contours, block fill, contour lines, colorbar rendering, axis labelling, and title placement — all in one flat function, with the map/non-map paths duplicated in parallel `if ptype == 1 / elif ptype == 0` blocks.

Proposed class structure

```
python

# contour.py

class ContourLayout:
    """
    Manages the figure geometry: the viewport (axes where data is drawn)
    and the surrounding margin space reserved for annotations
    (title, axis labels, colorbar, dimension titles).

    Separating this from rendering logic means layout can be inspected,
    adjusted, or reused without re-running data processing.
    """
    def __init__(self, plotvars):
        self.viewport: Axes # the data axes
        self.colorbar_ax: Axes # allocated below/beside the viewport
        self.title_ax: Axes # space above for title + dimension titles
        self._plotvars = plotvars

    def allocate(self, colorbar_orientation, colorbar_position, ...):
        """Reserve space for annotations; set self.viewport."""
        ...

    def apply_title(self, title, dims_title, fontsize, fontweight): ...
    def apply_axis_labels(self, xlabel, ylabel, xticks, yticks, ...): ...

class ContourData:
    """
    Holds the extracted, validated, and pre-processed arrays ready
    for rendering. Constructed from either a cf.Field or raw numpy arrays.
    """
    field: np.ndarray
    x: np.ndarray
    y: np.ndarray
    ptype: int # 0-6
    colorbar_title: str
    xlabel: str
    ylabel: str
    levels: np.ndarray
    mult: int
    fmult: float
    irregular: bool
    is_ugrid: bool
    is_orca: bool

    @classmethod
    def from_cf_field(cls, f, colorbar_title, plotvars, verbose): ...

    @classmethod
    def from_arrays(cls, field, x, y): ...

    def _apply_orca_fix(self): ...
    def _window_to_map_extent(self, lonmin, lonmax, latmin, latmax): ...
    def _add_cyclic(self): ...

class ColourScale:
    """
    Encapsulates colour scale selection, level-to-colour mapping,
    and colorbar label generation. Replaces the scattered cscale_flag
    branching throughout con().
    """
    def __init__(self, plotvars):
        self._plotvars = plotvars

    def fit_to_levels(self, levels, includes_zero, levels_extend): ...
    def get_cmap(self) -> matplotlib.colors.ListedColormap: ...
    def colorbar_labels(self, levels, orientation, n_columns,
                        label_skip, custom_labels) -> list[str]: ...

class ContourRenderer:
    """
    Does the actual matplotlib/cartopy drawing into a ContourLayout viewport.
    One render call per layer (filled, blockfill, lines).
    Knows nothing about cf fields or level calculation.
    """
    def __init__(self, layout: ContourLayout, data: ContourData,
                 colour_scale: ColourScale):
        self.layout = layout
        self.data = data
        self.cs = colour_scale

    def render_filled(self, alpha, zorder, transform_first): ...
    def render_blockfill(self, fast, alpha, zorder): ...
    def render_lines(self, colors, linewidths, linestyles,
                    line_labels, zero_thick): ...
    def render_colorbar(self, orientation, shrink, position,
                       fraction, thick, anchor,
                       fontsize, fontweight,
                       text_up_down, text_down_up, drawedges): ...

class MapContourRenderer(ContourRenderer):
    """Map (ptype==1) specialisation — adds Cartopy transform,
    coastlines, and polar-projection handling."""
    def render_filled(self, ...): ... # wraps contourf with transform=
    def render_blockfill(self, ...): ...

class XYContourRenderer(ContourRenderer):
    """Non-map (ptype 0,2-5) specialisation — plain matplotlib axes."""
    ...

def con(f=None, x=None, y=None, **kwargs):
    """
    Public entry point — thin orchestrator.
    Builds the objects, calls them in order, returns None.
    """
    data = ContourData.from_cf_field(f, ...) if isinstance(f, cf.Field) \
        else ContourData.from_arrays(f, x, y)
    layout = ContourLayout(plotvars)
    layout.allocate(...)
    scale = ColourScale(plotvars).fit_to_levels(data.levels, ...)

    renderer_cls = MapContourRenderer if data.ptype == 1 else XYContourRenderer
    renderer = renderer_cls(layout, data, scale)

    if data.fill: renderer.render_filled(...)
    if data.blockfill: renderer.render_blockfill(...)
    if data.lines: renderer.render_lines(...)
    if data.colorbar: renderer.render_colorbar(...)

    layout.apply_title(...)
    layout.apply_axis_labels(...)
```

Key design decisions

ContourLayout owns the viewport/margin split. The current code has colorbar placement, title offsets, and axis label nudging scattered across hundreds of lines. Centralising this means you can reason about "what space is left for data" vs "what space is annotation" in one place.

ContourData is a value object — it's constructed once, validated once (`_check_data`, `orca_check`, `check_well_formed`), and then read-only. The current code mutates `field`, `x`, `y` in-place repeatedly (cyclic wrap, orca fix, polar subsetting) making it hard to reason about state.

ColourScale eliminates the `cscale_flag` integer enum. The `0/1/2` flag scattered with `if plotvars.cscale_flag == 0:` branches in four different places is a classic maintainability hazard. A proper object with a `fit_to_levels()` method makes the intent explicit.

MapContourRenderer vs **XYContourRenderer** via inheritance keeps the map/non-map duplication out of a single function while sharing `render_colorbar` and `render_lines` logic in the base class.

Want me to implement any of these classes in full, or start with a particular piece (e.g. **ContourLayout** since that's your key concern about viewport vs annotation space)?

